Software Design and Architecture and Principles

- Design is a meaningful engineering representation of something that is to be built.
- Software design is an iterative process through which requirements are translated into a “blueprint” for constructing the software.
- • Initially, the blueprint depicts a holistic view of software.

- That is, the design is represented at a high level of abstraction— a level that can be directly traced to the specific system objective and more detailed data, functional, and behavioral requirements.
- As design iterations occur, subsequent refinement leads to design representations at much lower levels of abstraction.
- These can still be traced to requirements, but the connection is more subtle

Design Guidelines

- Design should exhibit an architectural structure .
- A design should be modular logically , function& Sub functions
- A design should contain distinct representation of data, architecture, interfaces and components
- A design should lead to component that exhibit independent functional characteristics

DESIGN PRINCIPLES

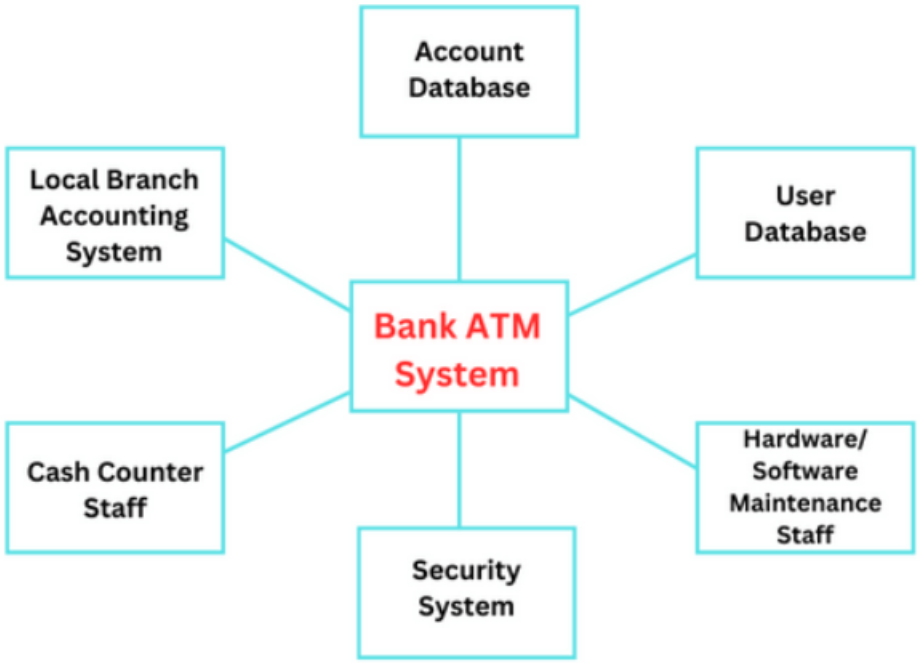
- The design process should not suffer from "tunnel vision" – alternative approach
- The design should be traceable to the analysis model.
- The design should not reinvent the wheel-- > already exist (new ideas)
- The design should "minimize the intellectual distance" between the software and the problem

DESIGN PRINCIPLES

- The design should exhibit uniformity and integration.– rules and format should be defined for a design team
- The design should be structured to accommodate change.
- The design should be structured to degrade gently, even when aberrant data, events, or operating conditions are encountered.

Context Models

- The external perspective model represents how the system that must be built will interact with other systems within its environment.
- It shows the boundaries of a system that includes various automated systems in an environment.
- The requirements of employees and stakeholders are discussed while creating the context model to decide the system's functionality that needs to be developed.
- It shows the relationships between the system to be created and other systems.


Account Database
Local Branch Accounting System
User Database
Bank ATM System
Cash Counter Staff
Hardware/ Software Maintenance Staff
Security System

Architectural Design

- Architectural design is concerned with understanding how a system should be organized and designing the overall structure of that system.
- Architectural design is the first stage in the software design process.
- It is the critical link between design and requirements engineering, as it identifies the main structural components in a system and the relationships between them.

Architectural Design

- The output of the architectural design process is an architectural model that describes how the system is organized as a set of communicating components.
- Software architectures at two levels of abstraction

1. Architecture in the small
2. Architecture in the large

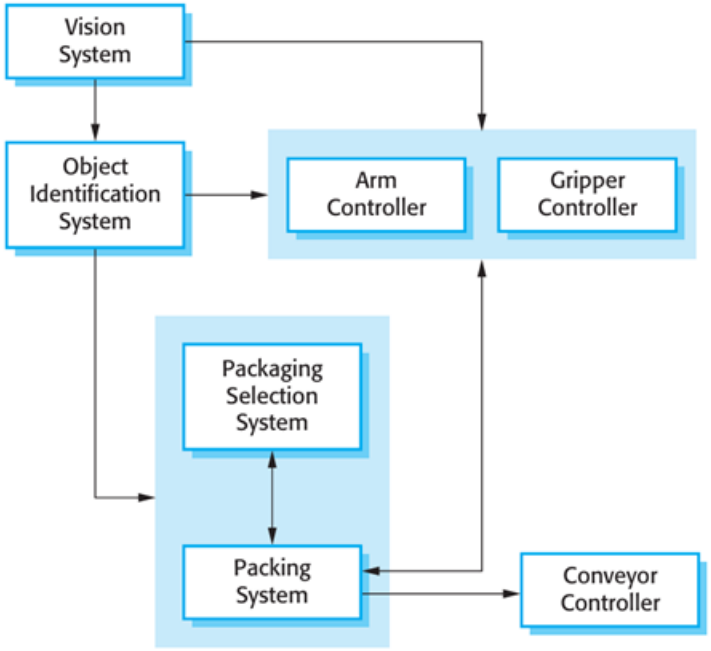


Figure 6.1 The architecture of a packing robot control system

Architectural design decisions

- Architectural design is a creative process where you design a system organization that will satisfy the functional and non-functional requirements of a system.
- Because it is a creative process, the activities within the process depend on the type of system being developed, the background and experience of the system architect, and the specific requirements for the system.
- It is therefore useful to think of architectural design as a series of decisions to be made rather than a sequence of activities.

Architectural design decisions

1. Is there a generic application architecture that can act as a template for the sys tem that is being designed?
2. How will the system be distributed across a number of cores or processors?
3. What architectural patterns or styles might be used?
4. What will be the fundamental approach used to structure the system?
5. How will the structural components in the system be decomposed into sub components?

Architectural design decisions

6. What strategy will be used to control the operation of the components in the system?

7. What architectural organization is best for delivering the non-functional requirements of the system?

8. How will the architectural design be evaluated?

9. How should the architecture of the system be documented

- The layered architecture pattern is another way of achieving separation and independence.
- This pattern is shown in Figure below.
- The system functionality is organized into separate layers, and each layer only relies on the facilities and services offered by the layer immediately beneath it.

Name	Layered architecture
Description	Organizes the system into layers with related functionality associated with each layer. A layer provides services to the layer above it so the lowest-level layers represent core services that are likely to be used throughout the system. See Figure 6.6.
Example	A layered model of a system for sharing copyright documents held in different libraries, as shown in Figure 6.7.
When used	Used when building new facilities on top of existing systems; when the development is spread across several teams with each team responsibility for a layer of functionality; when there is a requirement for multi-level security.
Advantages	Allows replacement of entire layers so long as the interface is maintained. Redundant facilities (e.g., authentication) can be provided in each layer to increase the dependability of the system.
Disadvantages	In practice, providing a clean separation between layers is often difficult and a high-level layer may have to interact directly with lower-level layers rather than through the layer immediately below it. Performance can be a problem because of multiple levels of interpretation of a service request as it is processed at each layer.

This shows the organization of the system for mental health care (MHC-PMS)

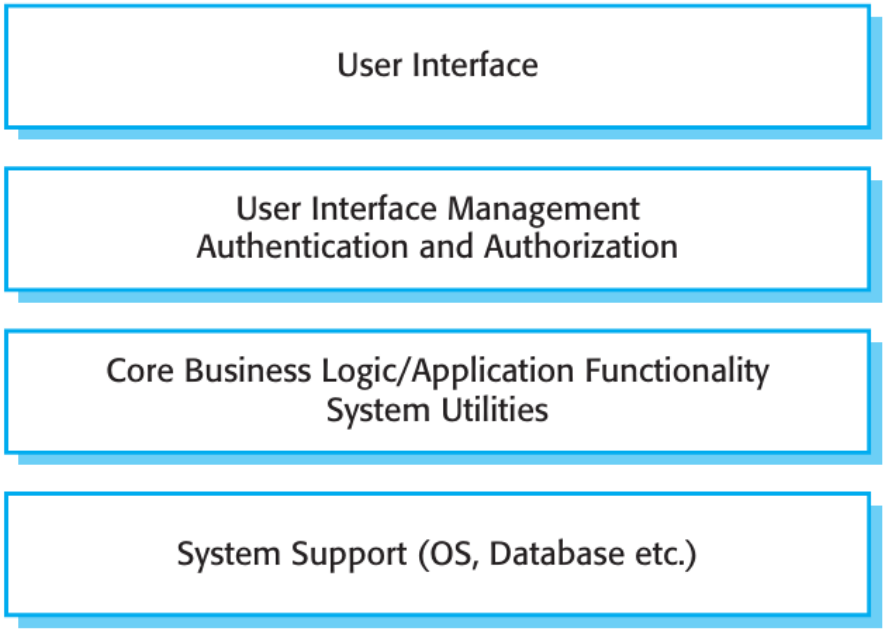


Figure 6.6 A generic layered architecture

- The layered architecture and MVC patterns are examples of patterns where the view presented is the conceptual organization of a system.

Name	Repository
Description	All data in a system is managed in a central repository that is accessible to all system components. Components do not interact directly, only through the repository.
Example	Figure 6.9 is an example of an IDE where the components use a repository of system design information. Each software tool generates information which is then available for use by other tools.
When used	You should use this pattern when you have a system in which large volumes of information are generated that has to be stored for a long time. You may also use it in data-driven systems where the inclusion of data in the repository triggers an action or tool.
Advantages	Components can be independent—they do not need to know of the existence of other components. Changes made by one component can be propagated to all components. All data can be managed consistently (e.g., backups done at the same time) as it is all in one place.
Disadvantages	The repository is a single point of failure so problems in the repository affect the whole system. May be inefficiencies in organizing all communication through the repository. Distributing the repository across several computers may be difficult.

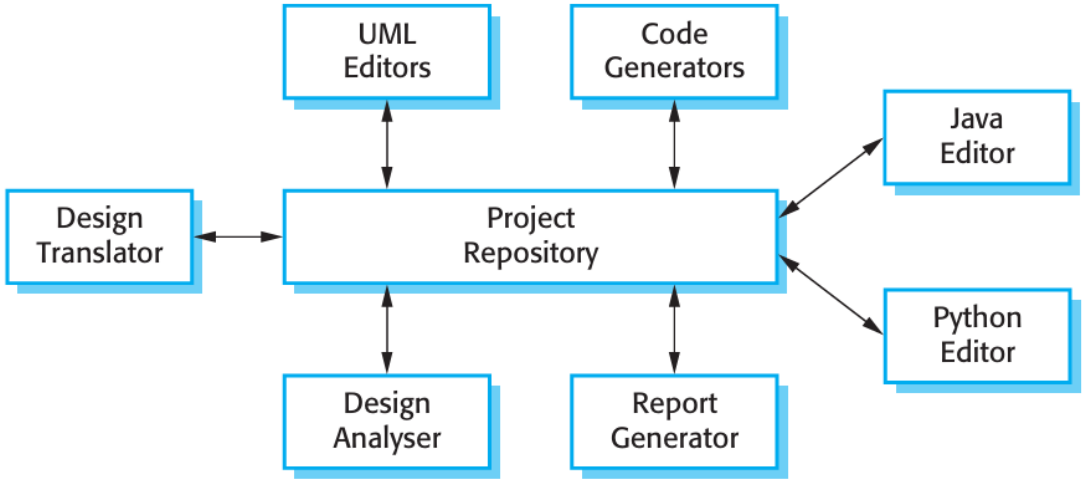


Figure 6.9 A repository architecture for an IDE

- The repository pattern is concerned with the static structure of a system and does not show its run-time organization.

Name	Client–server
Description	In a client–server architecture, the functionality of the system is organized into services, with each service delivered from a separate server. Clients are users of these services and access servers to make use of them.
Example	Figure 6.11 is an example of a film and video/DVD library organized as a client–server system.
When used	Used when data in a shared database has to be accessed from a range of locations. Because servers can be replicated, may also be used when the load on a system is variable.
Advantages	The principal advantage of this model is that servers can be distributed across a network. General functionality (e.g., a printing service) can be available to all clients and does not need to be implemented by all services.
Disadvantages	Each service is a single point of failure so susceptible to denial of service attacks or server failure. Performance may be unpredictable because it depends on the network as well as the system. May be management problems if servers are owned by different organizations.

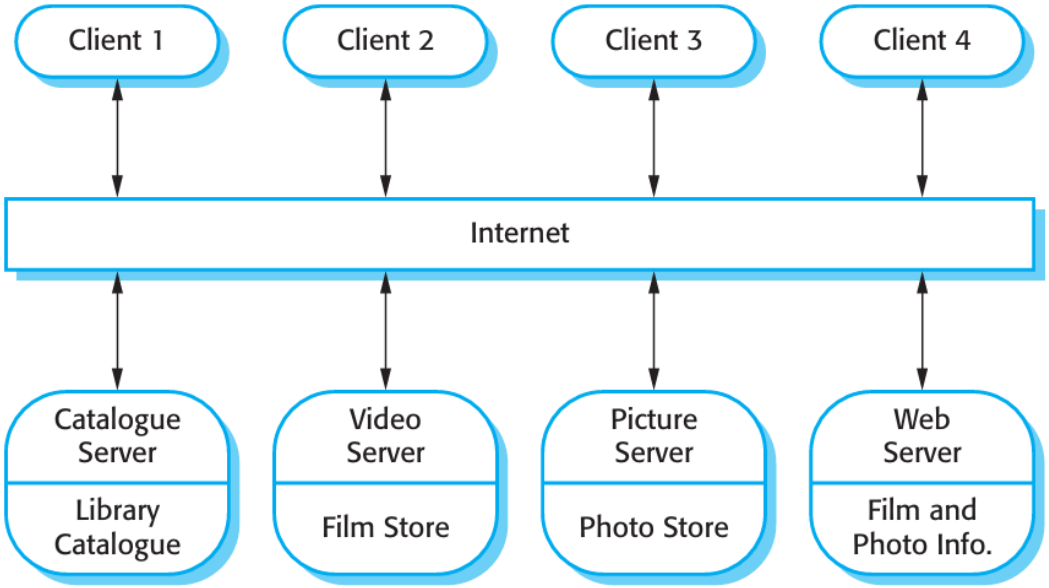
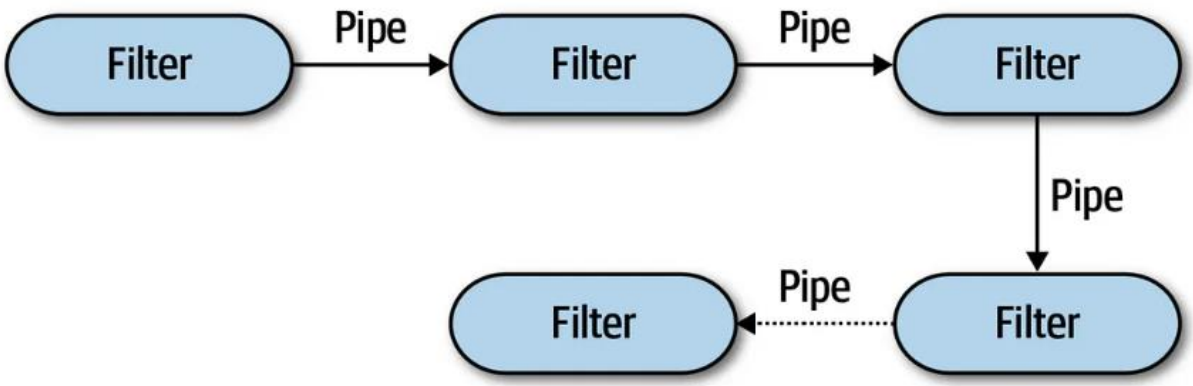
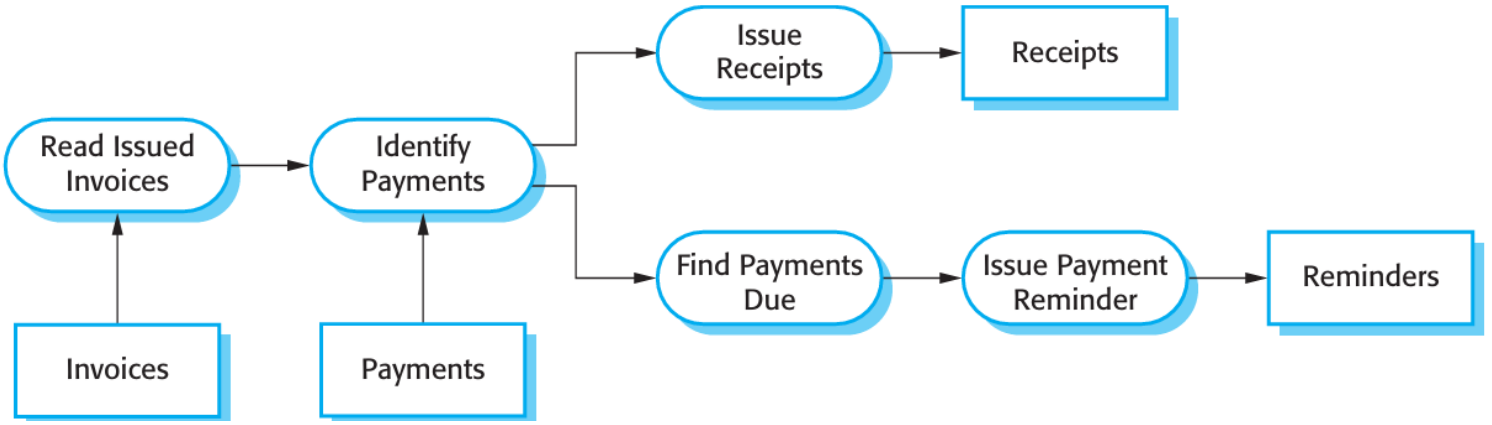


Figure 6.11 A client–server architecture for a film library

Name	Pipe and filter
Description	The processing of the data in a system is organized so that each processing component (filter) is discrete and carries out one type of data transformation. The data flows (as in a pipe) from one component to another for processing.
Example	Figure 6.13 is an example of a pipe and filter system used for processing invoices.
When used	Commonly used in data processing applications (both batch- and transaction-based) where inputs are processed in separate stages to generate related outputs.
Advantages	Easy to understand and supports transformation reuse. Workflow style matches the structure of many business processes. Evolution by adding transformations is straightforward. Can be implemented as either a sequential or concurrent system.
Disadvantages	The format for data transfer has to be agreed upon between communicating transformations. Each transformation must parse its input and unparse its output to the agreed form. This increases system overhead and may mean that it is impossible to reuse functional transformations that use incompatible data structures.


Filter
Pipe
Filter
Pipe
Filter
Pipe
Filter
Pipe
Filter


Read Issued Invoices
Identify Payments
Issue Receipts
Receipts
Find Payments Due
Issue Payment Reminder
Reminders
Invoices
Payments

Design Patterns

- The pattern is a description of the problem and the essence of its solution, so that the solution may be reused in different settings.
- The pattern is not a detailed specification. Rather, you can think of it as a description of accumulated wisdom and experience, a well-tried solution to a com mon problem.

Design Pattern

Pattern name: Observer

Description: Separates the display of the state of an object from the object itself and allows alternative displays to be provided. When the object state changes, all displays are automatically notified and updated to reflect the change.

Problem description: In many situations, you have to provide multiple displays of state information, such as a graphical display and a tabular display. Not all of these may be known when the information is specified. All alternative presentations should support interaction and, when the state is changed, all displays must be updated.

This pattern may be used in all situations where more than one display format for state information is required and where it is not necessary for the object that maintains the state information to know about the specific display formats used.

Solution description: This involves two abstract objects, Subject and Observer, and two concrete objects, ConcreteSubject and ConcreteObject, which inherit the attributes of the related abstract objects. The abstract objects include general operations that are applicable in all situations. The state to be displayed is maintained in ConcreteSubject, which inherits operations from Subject allowing it to add and remove Observers (each observer corresponds to a display) and to issue a notification when the state has changed.

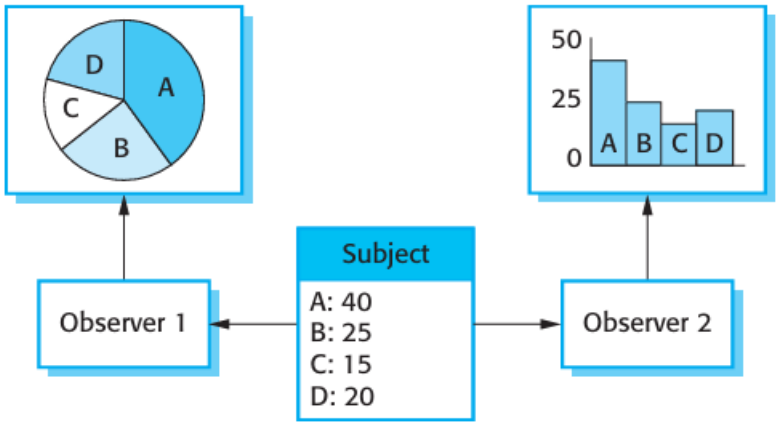
The ConcreteObserver maintains a copy of the state of ConcreteSubject and implements the Update() interface of Observer that allows these copies to be kept in step. The ConcreteObserver automatically displays the state and reflects changes whenever the state is updated.
The UML model of the pattern is shown in Figure 7.12.

Consequences: The subject only knows the abstract Observer and does not know details of the concrete class. Therefore there is minimal coupling between these objects. Because of this lack of knowledge, optimizations that enhance display performance are impractical. Changes to the subject may cause a set of linked updates to observers to be generated, some of which may not be necessary.

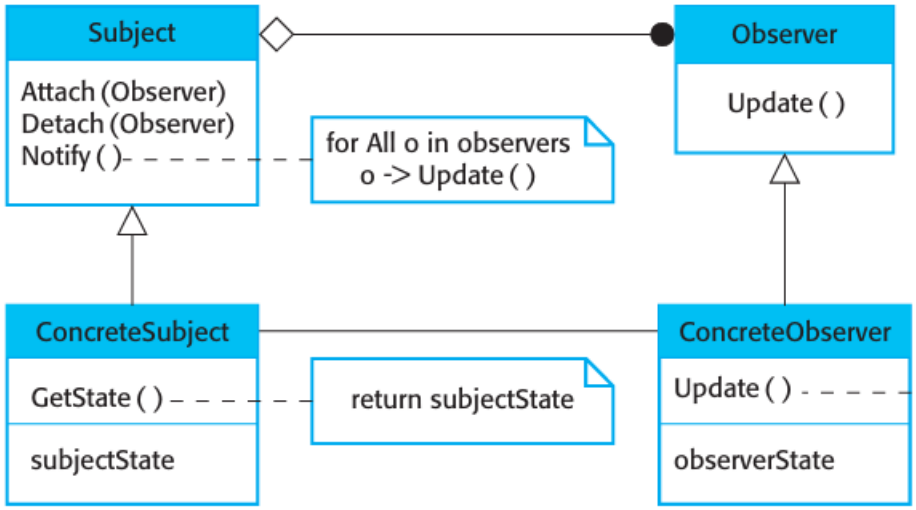
Essential elements of design patterns

1. A **name** that is a meaningful reference to the pattern.
2. A **description of the problem** area that explains when the pattern may be applied.
3. A **solution description** of the parts of the design solution, their relationships, and their responsibilities. This is not a concrete design description. It is a template for a design solution that can be instantiated in different ways. This is often expressed graphically and shows the relationships between the objects and object classes in the solution.
4. A **statement of the consequences**—the results and trade-offs—of applying the pattern. This can help designers understand whether or not a pattern can be used in a particular situation

Observer Design Pettern



Observer Design Pattern representation in UML



Different types of Design Pattern

1.Tell several objects that the state of some other object has changed (Observer pattern).

2. Tidy up the interfaces to a number of related objects that have often been devel oped incrementally (Façade pattern).

3. Provide a standard way of accessing the elements in a collection, irrespective of how that collection is implemented (Iterator pattern).

4. Allow for the possibility of extending the functionality of an existing class at run-time (Decorator pattern).

Advantages of using design patterns:

- **Reusability**: Design patterns enable developers to reuse proven solutions rather than starting from scratch. This can accelerate the development process and reduce the likelihood of errors.
- **Maintainability**: Code that follows design patterns is often easier to maintain. Patterns provide a clear structure and organization, making it simpler to update or modify the code as requirements change.
- **Scalability**: Many design patterns are designed with scalability in mind. They help in creating systems that can easily grow and adapt to increased load or complexity without requiring a complete rewrite.
- **Best Practices**: Design patterns encapsulate best practices in software design. They reflect the experience of many developers and can guide teams in creating high-quality, robust software.

Advantages of Design pattern

- **Flexibility and Extensibility**: Many design patterns promote loose coupling and high cohesion, which allows for greater flexibility and easier extension of software systems. This makes it simpler to add new features or modify existing ones without affecting unrelated components.
- **Documentation**: Design patterns often come with well-documented examples and explanations, which can serve as a valuable reference for developers. This documentation can help in understanding the rationale behind certain design choices.
- **Error Reduction**: By following established patterns, developers can avoid common pitfalls and anti-patterns, leading to fewer bugs and issues in the code.
- **Enhanced Collaboration**: Teams can work more effectively when they share a common understanding of design patterns. This can lead to more cohesive design decisions and a more integrated product.

Security by Design

- Security by design is a fundamental principle in software engineering that emphasizes the importance of incorporating security measures and considerations into the software development process from the very beginning, rather than treating security as an afterthought or a secondary concern.
- This approach aims to create secure software systems by addressing potential vulnerabilities and threats during the design and development phases rather than waiting until the software is completed or already in use.

Security by Design Principles

- **Threat Modeling**: Identify potential threats and vulnerabilities in the early stages of development. This includes understanding the system architecture, potential attack vectors, and the impact of different types of attacks.
- **Secure Coding Practices**: Encourage developers to follow coding standards that minimize vulnerabilities such as SQL injection, cross-site scripting (XSS), and buffer overflow attacks. This includes input validation, output encoding, and error handling.
- **Regular Security Testing**: Integrate security testing into the software development lifecycle, including static code analysis, dynamic analysis, and penetration testing. Continuous integration/continuous deployment (CI/CD) pipelines can include automated security checks.
- **User Education and Awareness**: Educate users about security best practices and potential threats. This includes training on recognizing phishing attacks and understanding the importance of strong passwords.
- **Compliance and Standards**: Adhere to relevant security standards and regulations (such as GDPR, HIPAA, or PCI-DSS) during the design and development phases to ensure legal and regulatory compliance.

Security by Design

- **Secure Architecture and Design Patterns**: Use established security architecture frameworks and design patterns that promote secure software design, such as the OWASP Secure Application Development Lifecycle framework.
- **Incident Response Planning**: Prepare for potential security incidents by having an incident response plan in place. This includes procedures for detecting, responding to, and recovering from security breaches.
- **Continuous Improvement**: Security is not a one-time effort. Organizations should continually assess and improve their security practices based on evolving threats and vulnerabilities.